

Godlog - May 10, 2026
Single-Detector LLR/FAR Proposal for SPIIR
Local implementation plan for the Eric branch

SPIIR workflow notes

May 10, 2026

This file records the proposed single-detector implementation for the Eric branch. The intent is:

- use the main SPIIR pipeline as the stable upstream coherent search;
- use the `wguo-single-detector` branch as the baseline for single-detector row extraction/routing;
- add the missing piece: a single-detector likelihood-ratio rank and FAR assignment path;
- keep the proposal focused on the single branch only.

Key point

The final single-detector target is not a raw (ρ_m, χ_r^2) plot. The target calibrated product is

$$(\rho_m, -\log_{10} \text{FAR}_{\text{sd}}).$$

May 10, 2026 – Code Bases to Combine in the Eric Branch

Source	Role in the realization
Main SPIIR	Provides the online/offline postcoh pipeline, detector conditioning, SPIIR filtering, <code>cuda_postcoh</code> , coherent background/FAR, and <code>FinalSink</code> .
<code>wguo-single-detector</code>	Provides the single-detector extraction idea: preserve detector-local information from postcoh/coexistence rows so H1 and L1 can be analyzed as independent single-detector categories.
Eric branch	Should combine the two bases and add the new single-detector LLR/FAR layer described here.

<https://git.ligo.org/lscsoft/spiir>

https://git.ligo.org/lscsoft/spiir/-/tree/wguo-single-detector?ref_type=heads

The local code in this workspace is not a Git checkout, so the changes are prepared locally for later push by the user.

Upstream products already available

- detector-local complex SNR streams;
- local peak SNR amplitudes `snglsnr[j]`;
- detector-local autochisq values `chisq[j]`;
- row flags: foreground, background, empty;
- bank/template/time metadata;
- coherent quantities such as `cohsnr`, `nullsnr`, and `cmbchisq`.

Single-side interpretation

$$\rho_m = \text{snglsnr}[j], \quad \chi_r^2 = \text{chisq}[j].$$

For each detector $j \in \{\text{H1}, \text{L1}\}$, one postcoh row can produce a detector-local feature

$$d_{j,m} = (\rho_m, \chi_r^2).$$

The current code can expose the needed observables, but the single branch still needs a complete statistical significance layer:

- 1 define H_0 and H_1 likelihoods for (ρ_m, χ_r^2) ;
- 2 compute a log-likelihood-ratio rank $r(d)$;
- 3 use direct rank-tail FAR only during the cold-start stage;
- 4 fit the bootstrap (r, FAR) relation after enough data;
- 5 persist that fitted relation in a dedicated single FAR–LLR file;
- 6 assign later FAR from the fitted LLR-to-FAR function;
- 7 output $(\rho_m, -\log_{10} \text{FAR}_{\text{sd}})$ rows.

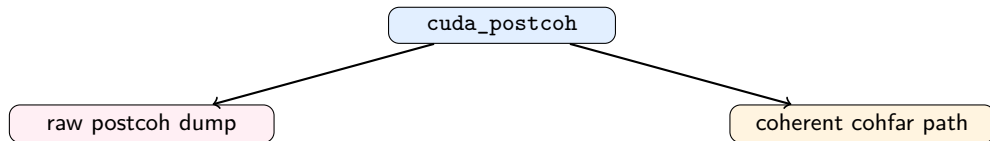
The single-detector background file is only the FAR–LLR calibration file. It stores the relation between LLR and FAR, for example (r, FAR) points plus a fitted function. It is not the coherent XML statistics file, not the zero-lag trigger stream, and not a persistent two-dimensional (ρ_m, χ_r^2) map.

May 10, 2026 – Single-Branch Boundary in the Pipeline

Code anchor: `spiir/gstlal-spiir/python/pipemodules/spiirparts.py; mkPostcohSPIIROnline(...)`, after `mkcudapostcoh(...)`

The natural branch point is after `cuda_postcoh`, because this is the first point where every postcoh row already contains:

`snglsnr[j]`, `chisq[j]`, `is_background`, `ifos`, `tmplt_idx`, `bankid`.



The single branch must not recompute matched filtering, $C_{j,m}$, `cohsnr`, or `cmbchisq`. It reuses the detector-local pair already created upstream.

May 10, 2026 – Current Local Hook in pipeline.sh

Code anchor: spiir/pipeline.sh; mode parser and post-graph single analyzer call

```
PIPELINE_MODE="${PIPELINE_MODE:-}"
...
export PIPELINE_MODE
"${base_cmd[@]}"

single_far_csv="${jobno}/${jobno}_single_detector_far.csv"
single_far_background_input="${SINGLE_FAR_BACKGROUND_INPUT:-}"
single_far_background_output="${SINGLE_FAR_BACKGROUND_OUTPUT:-${jobno}/${jobno}_single_far_llr_background.json}"
```

In `-single` mode, the coherent GStreamer graph still runs. After that graph exits, the offline single analyzer reads `sdpostcoh*.xml.gz` and writes single-detector FAR rows.

May 10, 2026 – Local Code Added to Launch LLR/FAR

Code anchor: `spiiir/pipeline.sh`; `single_detector_cmd`

```
single_detector_cmd=(
  python "${single_detector_py}" single
  --postcoh-glob "${jobno}/sdpostcoh*.xml.gz"
  --output "${single_far_csv}"
  --ifos H1,L1
  --min-snr 4
  --background-output "${single_far_background_output}"
  --snr-bins "${SINGLE_FAR_SNR_BINS:-4,6,8,12,inf}"
)
if [[ -n "${single_far_background_input}" ]]; then
  single_detector_cmd+=(--background-input "${single_far_background_input}")
fi
if [[ "${single_far_calibrate}" == "1" ]]; then
  single_detector_cmd+=(--calibrate-noise-dof)
fi
"${single_detector_cmd[@]}"
```

Cold start: if no `SINGLE_FAR_BACKGROUND_INPUT` is supplied, the run estimates FAR directly from the current bootstrap ranks for about one week, fits the resulting LLR–FAR relation, and writes the first `single_far_llr_background.json`. Later runs load that fitted relation, assign FAR from it, and update it.

The single branch only uses detector-local H1 and L1 information:

$$j \in \{\text{H1}, \text{L1}\}.$$

They are analyzed separately, not as a coherent HL network:

$$\text{H1} \rightarrow \text{H1_sd}, \quad \text{L1} \rightarrow \text{L1_sd}.$$

For each available detector,

$$\rho_m = \text{snglsnr}[j], \quad \chi_r^2 = \text{chisq}[j].$$

Key point

If only H1 or only L1 is available because the other detector loses lock, the available detector can still contribute to its own single-detector category.

A multi-detector coexistence row, for example HL or HLV, can still carry detector-local fields:

$$\text{snglsnr}[H1], \quad \text{chisq}[H1], \quad \text{snglsnr}[L1], \quad \text{chisq}[L1].$$

The single branch should copy the detector-local H1/L1 pairs into the single categories:

$$H1 : (\text{snglsnr}[H1], \text{chisq}[H1]) \rightarrow H1_sd,$$

$$L1 : (\text{snglsnr}[L1], \text{chisq}[L1]) \rightarrow L1_sd.$$

This is not numerical subtraction. The coherent row remains in the coherent branch; the single branch only duplicates the local H/L pair for independent single-detector ranking.

Code anchor: `spiir/gstlal-spiir/python/pipemodules/single_detector_far.py; features_from_postcoh_row`

For every postcoh row and every requested detector j , the analyzer reads:

$$\rho_m = \text{snglsnr}[j], \quad \chi_r^2 = \text{chisq}[j].$$

It rejects missing, nonpositive, or below-threshold values. The retained object is:

$$d_{j,m} = (\rho_m, \chi_r^2).$$

The single branch does not inspect the full SNR stream at this stage. It trusts `cuda_postcoh` to have already selected the trigger peak and computed the detector-local autochisq around that peak.

For fixed detector j and template m , SPIIR has a complex SNR time series:

$$\rho_{j,m}(t).$$

A trigger is centered at one detector-local peak time t_j . The complex peak value is

$$\hat{\rho}_{j,m} = \rho_{j,m}(t_j).$$

The single-detector SNR coordinate is the peak amplitude:

$$\rho_m = |\hat{\rho}_{j,m}| = \text{snglsnr}[j].$$

Key point

ρ_m is the peak SNR for this detector and this template. It is not `cohsnr`, because no network projection is used in the single branch.

The detector-local shape statistic compares the SNR samples around the peak to the stored local template shape:

$$r_{j,m}(\Delta) = \rho_{j,m}(t_j + \Delta) - \hat{\rho}_{j,m} C_{j,m}(\Delta), \quad \Delta = -L, \dots, L.$$

The reduced statistic is

$$\chi_r^2 \equiv \chi_{j,m}^2 = \frac{1}{N_{j,m}} \sum_{\Delta=-L}^L |r_{j,m}(\Delta)|^2.$$

In code this is stored as:

$$\chi_r^2 = \text{chisq}[j].$$

Small χ_r^2 means the local window is consistent with the template shape; large χ_r^2 is more glitch-like.

The new rank uses two statistical hypotheses:

H_0 noise/glitch hypothesis: the local peak is noise-like. The residual shape has no deterministic signal-like shift.

H_1 signal hypothesis: the local peak is signal-like. The residual may contain Gaussian noise plus a small deterministic template-mismatch shift.

The desired rank is a log-likelihood ratio:

$$r(d) \simeq \ln \frac{P(d \mid H_1)}{P(d \mid H_0)}, \quad d = (\rho_m, \chi_r^2).$$

Using the product rule:

$$r(d) = \ln \frac{P(\chi_r^2 \mid \rho_m, H_1)}{P(\chi_r^2 \mid \rho_m, H_0)} + \ln \frac{P(\rho_m \mid H_1)}{P(\rho_m \mid H_0)} + \text{const.}$$

In the ideal exact-match case,

$$\rho_{j,m}(t_j + \Delta) = \hat{\rho}_{j,m} C_{j,m}(\Delta) + n(\Delta),$$

so the residual is

$$r_{j,m}(\Delta) = n(\Delta).$$

After whitening, the local noise is modeled as Gaussian in the complex components. Therefore

$$|r_{j,m}(\Delta)|^2 = r_{\text{re}}(\Delta)^2 + r_{\text{im}}(\Delta)^2$$

is a sum of Gaussian squares, and the local quadratic power is chi-square-like.

The actual online trigger stream is peak-selected and clustered, so the ideal degree count is not trusted directly. We use an empirical effective degree of freedom ν_{eff} .

Under H_0 , there is no deterministic mismatch shift:

$$\lambda = 0.$$

The working model is a reduced central chi-square:

$$Y = \nu_{\text{eff}} \chi_r^2, \quad Y \mid H_0 \sim \chi_{\nu_{\text{eff}}}^2.$$

Therefore the density for the observed reduced statistic is

$$P(\chi_r^2 \mid \rho_m, H_0) = \nu_{\text{eff}} f_{\chi_{\nu_{\text{eff}}}^2}(\nu_{\text{eff}} \chi_r^2).$$

Key point

ν_{eff} is not calculated from a single trigger. It must be calibrated from many background/noise triggers in the same category, optionally in SNR bins.

For a background category, such as H1_sd, collect background triggers and read:

$$\chi_{r,k}^2 = \text{chisq}[j], \quad k = 1, \dots, N_{\text{bg}}.$$

If the reduced statistic follows

$$\chi_r^2 \sim \frac{1}{\nu_{\text{eff}}} \chi_{\nu_{\text{eff}}}^2,$$

then

$$\text{Var}(\chi_r^2) = \frac{2}{\nu_{\text{eff}}}.$$

So a simple calibration is

$$\nu_{\text{eff}} \approx \frac{2}{\text{Var}_{\text{bg}}(\chi_r^2)}.$$

The local code now supports SNR-bin calibration:

$$\rho_m \in [4, 6), [6, 8), [8, 12), [12, \infty).$$

Under H_1 , a real signal may not perfectly match the stored local template shape. Write

$$s_{j,m}(\Delta) = \hat{\rho}_{j,m} C_{j,m}(\Delta) (1 + \beta(\Delta)).$$

Then the tested residual becomes

$$r_{j,m}(\Delta) = n(\Delta) + \beta(\Delta) \hat{\rho}_{j,m} C_{j,m}(\Delta).$$

Within the short local window, approximate

$$\beta(\Delta) \approx \bar{\beta}.$$

Key point

The residual is Gaussian noise plus a deterministic mismatch shift. This is why the signal-side shape likelihood is modeled with a noncentral chi-square distribution.

Define the deterministic mismatch shift

$$\mu(\Delta) = \bar{\beta} \hat{\rho}_{j,m} C_{j,m}(\Delta).$$

For a sum of squared Gaussian variables with deterministic mean shift, the noncentrality is the squared norm of that shift:

$$\lambda = \sum_{\Delta=-L}^L |\mu(\Delta)|^2.$$

Substituting $\mu(\Delta)$ gives

$$\lambda(\bar{\beta}, \rho_m) = \bar{\beta}^2 \rho_m^2 \sum_{\Delta=-L}^L |C_{j,m}(\Delta)|^2.$$

λ is not a new measured feature. It is the noncentral chi-square parameter introduced by the likelihood model. It grows with mismatch level, peak SNR, and local template-shape power.

For fixed $\bar{\beta}$, use

$$Y = \nu_{\text{eff}} \chi_r^2, \quad Y \mid (\rho_m, \bar{\beta}, H_1) \sim \chi_{\nu_{\text{eff}}}^2(\lambda).$$

Thus

$$P(\chi_r^2 \mid \rho_m, \bar{\beta}, H_1) = \nu_{\text{eff}} f_{\chi_{\nu_{\text{eff}}}^2(\lambda)}(\nu_{\text{eff}} \chi_r^2).$$

Because the true $\bar{\beta}$ is unknown event by event, treat it as a nuisance parameter:

$$\bar{\beta} \sim \text{Uniform}(0, \beta_{\text{max}}).$$

The final signal-side likelihood averages over allowed mismatch levels:

$$P(\chi_r^2 \mid \rho_m, H_1) = \frac{1}{\beta_{\max}} \int_0^{\beta_{\max}} \nu_{\text{eff}} f_{\chi_{\nu_{\text{eff}}}^2}(\lambda(\bar{\beta}, \rho_m)) (\nu_{\text{eff}} \chi_r^2) d\bar{\beta}.$$

In the code this is evaluated by a discrete beta grid:

$$P(\chi_r^2 \mid \rho_m, H_1) \approx \sum_i w_i g_i(\chi_r^2; \rho_m, \beta_i), \quad \sum_i w_i = 1.$$

The prototype default uses $\beta_{\max} = 0.03$ and a uniform beta grid. The grid and weights are implementation choices that should later be calibrated or validated with injections/background.

The full single-detector rank is

$$r(d) = \ln \frac{P(\chi_r^2 | \rho_m, H_1)}{P(\chi_r^2 | \rho_m, H_0)} + \frac{1}{2}\rho_m^2 + \text{const.}$$

The term $+\rho_m^2/2$ is the SNR-amplitude contribution from the matched-filter likelihood after maximizing over amplitude/phase/time.

For ranking, any fixed prior odds only add a constant:

$$\ln \frac{P(H_1 | d)}{P(H_0 | d)} = r(d) + \ln \frac{P(H_1)}{P(H_0)}.$$

Therefore $r(d)$ is enough to order events.

Code anchor: `spiir/gstlal-spiir/python/pipemodules/single_detector_far.py`; `SingleDetectorLikelihoodModel`

```
def log_signal_shape_pdf(self, rho, chisq, autocorr_power=None, ifo=None):
    dof = self.signal_dof_for(rho, ifo)
    x = dof * float(chisq)
    terms = []
    for beta, weight in zip(self.beta_grid, self.beta_weights):
        lam = self.noncentrality(rho, beta, autocorr_power)
        terms.append(math.log(weight) + noncentral_chisq_logpdf(
            x, dof, lam, max_terms=self.ncx2_max_terms))
    return math.log(dof) + logsumexp(terms)

def log_noise_shape_pdf(self, rho, chisq, ifo=None):
    dof = self.noise_dof_for(rho, ifo)
    x = dof * float(chisq)
    return math.log(dof) + central_chisq_logpdf(x, dof)
```

The factor $\log(\nu_{\text{eff}})$ is the Jacobian from $Y = \nu_{\text{eff}} \chi_r^2$.

Code anchor: `spiir/gstlal-spiir/python/pipemodules/single_detector_far.py`; `log_likelihood_ratio` and `rank`

```
def log_likelihood_ratio(self, rho, chisq, autocorr_power=None, ifo=None):
    shape_llr = (
        self.log_signal_shape_pdf(rho, chisq, autocorr_power, ifo=ifo)
        - self.log_noise_shape_pdf(rho, chisq, ifo=ifo))
    snr_llr = self.snr_log_weight * float(rho) * float(rho)
    return shape_llr + snr_llr + self.rank_offset

def rank(self, rho, chisq, autocorr_power=None, ifo=None):
    return self.log_likelihood_ratio(rho, chisq, autocorr_power, ifo=ifo)
```

This turns every single-detector feature $d = (\rho_m, \chi_r^2)$ into one scalar r . Larger r means more signal-like.

For every single-detector background trigger, compute

$$r_k^{(\text{bg})} = r(d_k^{(\text{bg})}).$$

This rank list is needed at the beginning because there is not yet a single-detector FAR–LLR file. During this bootstrap stage, for a foreground candidate with rank r_* , count same-category background triggers with rank at least as large:

$$N_{\text{bg}}^{(j)}(\geq r_*) = \sum_{k=1}^{N_{\text{bg}}^{(j)}} \mathbf{1}(r_k^{(\text{bg},j)} \geq r_*).$$

The indicator function is one if the condition is true and zero otherwise.

Key point

This direct counting is a bootstrap tool, not the final production FAR assignment. It creates the first data needed to fit the LLR–FAR relation.

Let $T_{\text{bg}}^{(j)}$ be the background livetime for the same detector category. Before a FAR–LLR file exists, the provisional direct FAR is

$$\text{FAR}_{\text{direct}}^{(j)}(r_*) = \frac{N_{\text{bg}}^{(j)}(\geq r_*)}{T_{\text{bg}}^{(j)}}.$$

These direct values provide calibration points

$$(r_*, \text{FAR}_{\text{direct}}^{(j)}(r_*))$$

for fitting the single-detector FAR–LLR file.

H1 and L1 must keep separate bootstrap rank lists and separate livetimes. This stage lasts about one week in the intended workflow, only to initialize the background file.

After the bootstrap stage, the single-detector background file stores a fitted mapping

$$\text{FAR}_{\text{sd}}^{(j)} = F_{\text{fit}}^{(j)}(r).$$

For a new foreground trigger:

$$d_* = (\rho_m, \chi_r^2) \implies r_* = r(d_*) \implies \text{FAR}_{\text{sd}}^{(j)} = F_{\text{fit}}^{(j)}(r_*).$$

The final plotted coordinate is therefore

$$(\rho_m, -\log_{10} \text{FAR}_{\text{sd}}^{(j)}).$$

Key point

This is the normal post-bootstrap path. The final FAR is given by the fitted function, not by re-counting the trigger background from scratch for every event.

Code anchor: `spiir/gstlal-spiir/python/pipemodules/single_detector_far.py`; RankBackground

```
class RankBackground(object):
    def __init__(self):
        self._ranks = []
        self.livetime = 0.0

    def add_rank(self, rank):
        bisect.insort(self._ranks, float(rank))

    def count_ge(self, rank):
        idx = bisect.bisect_left(self._ranks, float(rank))
        return len(self._ranks) - idx

    def far(self, rank):
        if self.livetime <= 0.0:
            return 0.0
        return self.count_ge(rank) / self.livetime
```

This exact sorted-list version is suitable for offline cold-start validation. A production version should use the sorted ranks to build (r, FAR) calibration points and then persist the fitted LLR–FAR mapping.

Code anchor: `spiiir/gstlal-spiir/python/pipemodules/single_detector_far.py`; `SingleFarLlrBackgroundFile`

The single background file stores only what FAR assignment needs:

- detector categories, e.g. `H1_sd`, `L1_sd`;
- bootstrap calibration points (r_i, FAR_i) ;
- the fitted mapping $F_{\text{fit}}^{(j)}(r)$;
- the likelihood/calibration model used to compute those ranks.

$$\mathcal{H}_j = \left\{ (r_i^{(j)}, \text{FAR}_i^{(j)}); F_{\text{fit}}^{(j)}; \nu_{\text{eff}}(\rho_m) \right\}.$$

Key point

This file is the single-detector FAR–LLR file. It is not a coherent-style `cohfar` XML statistics file and it is not the trigger-background stream itself.

May 10, 2026 – Cold Start and Real-Time Update Logic

At the beginning, no single FAR–LLR background file exists. The first stage is a bootstrap:

- 1 run the single analyzer over about one week of data;
- 2 collect background rows and empty-row livetime;
- 3 calibrate ν_{eff} from background χ_r^2 ;
- 4 compute ranks for background and foreground rows;
- 5 assign provisional FAR directly from the in-run rank tail;
- 6 collect $(r, \text{FAR}_{\text{direct}})$ points;
- 7 fit a smooth or monotonic $F_{\text{fit}}(r)$;
- 8 write the first single FAR–LLR background file.

After that:

- 1 load the existing single FAR–LLR background file;
- 2 compute r_* for each new foreground candidate;
- 3 assign $\text{FAR} = F_{\text{fit}}(r_*)$;
- 4 update the LLR–FAR calibration with new background information;
- 5 write an updated background file snapshot.

May 10, 2026 – Persistent Background JSON

Code anchor: `spiiir/gstlal-spiir/python/pipemodules/single_detector_far.py; SingleFarLlrBackgroundFile.dump`

```
data = {  
    "version": self.VERSION,  
    "created_utc": datetime.datetime.utcnow().isoformat() + "Z",  
    "description": "single-detector FAR-LLR fit background",  
    "ifos": list(self.ifos),  
    "likelihood_model": self.model.to_dict(),  
    "llr_far_points": ...,  
    "far_fit_model": ...,  
    "fit_metadata": ...,  
}
```

This makes the file self-consistent: the same likelihood calibration used to create the LLR values can be reloaded before evaluating the FAR fit in later runs.

The proposal separates two related but different background tasks:

Feature-space calibration

Use background (ρ_m, χ_r^2) points to fit or validate model parameters:

$$\nu_{\text{eff}}(\rho_m), \quad \beta_{\text{max}}, \quad \text{tail behavior.}$$

This is for likelihood calibration, not direct FAR lookup.

Rank-space FAR assignment

During cold start, map every background trigger to r , then assign provisional FAR from the one-dimensional rank tail:

$$(\rho_m, \chi_r^2) \rightarrow r \rightarrow \frac{N_{\text{bg}}(\geq r_*)}{T_{\text{bg}}}.$$

These provisional points initialize $F_{\text{fit}}(r)$. After that, new triggers use

$$(\rho_m, \chi_r^2) \rightarrow r_* \rightarrow \text{FAR} = F_{\text{fit}}(r_*).$$

Row type	Single-branch action
FOREGROUND	Extract H1/L1 local features, compute r_* , then assign FAR through $F_{\text{fit}}(r_*)$ after the FAR–LLR file exists. During bootstrap only, assign provisional direct FAR.
BACKGROUND	Extract H1/L1 local features and compute r_{bg} . These ranks build the cold-start direct FAR points and later update the LLR–FAR calibration.
EMPTY	Increment category lifetime $T_{\text{bg}}^{(j)}$. These rows are essential for direct bootstrap FAR because FAR is a rate, not just a probability.

The key distinction is terminology: zero-lag foreground and time-shift background are trigger rows; the single-detector background file is the fitted LLR–FAR calibration product.

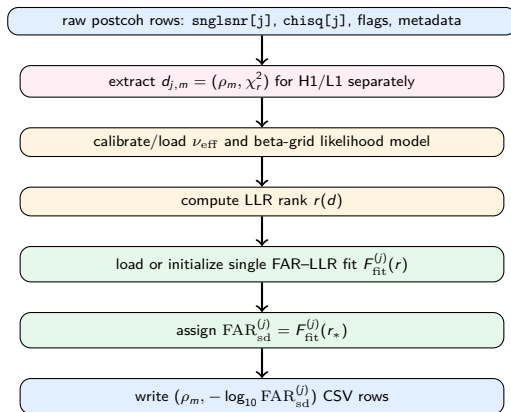
May 10, 2026 – Implementation Files Modified Locally

File	Local change
<code>spiir/pipeline.sh</code>	Adds single FAR–LLR input/output environment variables, cold-start calibration switch, and passes background options to the single analyzer.
<code>spiir/gstlal-spiir/python/ pipemodules/single_detector_far.py</code>	Adds persistent background file IO, ν_{eff} calibration, reduced chi-square likelihood scaling, beta marginalization controls, rank background, and FAR assignment.
<code>spiir/gstlal-spiir/python/ pipemodules/Makefile.am</code>	Installs <code>single_detector_far.py</code> and <code>combine_background_far.py</code> with the other pipemodules.

Key point

These changes are local. The user can push them to the GitLab Eric branch later.

May 10, 2026 – Expected Single-Branch Data Flow



The single branch needs validation in increasing order of risk:

- ➊ **Syntax and import checks:** `python -m py_compile` for the new Python files; `bash -n pipeline.sh`.
- ➋ **Unit checks:** central chi-square logpdf, noncentral Poisson-mixture logpdf, beta-mixture normalization, monotonic rank-tail count.
- ➌ **Toy rows:** fake postcoh rows with known foreground/background/empty behavior.
- ➍ **Small OzSTAR segment:** run `bash pipeline.sh -single` on a short known-good frame/cache segment.
- ➎ **Cold-start week:** build direct (r , FAR) points, fit F_{fit} , and write the first `single_far_llr_background.json`.
- ➏ **History-file run:** rerun with `SINGLE_FAR_BACKGROUND_INPUT` and confirm FAR assignment uses the loaded fit while updating the calibration snapshot.

- Check that the background χ_r^2 distribution is close enough to the fitted reduced chi-square model in each SNR bin.
- Check whether ν_{eff} depends strongly on detector, bank, template duration, or SNR.
- Validate β_{max} and the beta prior with injections; the current uniform prior is a modeling convention.
- Verify that H1 and L1 single categories have independent livetime accounting.
- Confirm that copied H/L detector-local fields from coexistence rows are not removed from the coherent branch.
- Decide how to handle zero exceedance counts: empirical upper limit $1/T_{\text{bg}}$ or a fitted tail extrapolation.

If an old background file contains ranks computed with one likelihood calibration, and the new run changes ν_{eff} , beta grid, or SNR weight, then old and new ranks are not directly comparable.

The local implementation handles this by storing the likelihood model in the FAR–LLR background JSON.

Recommended production policy:

- 1 if SINGLE_FAR_BACKGROUND_INPUT is used, load its likelihood model and do not recalibrate by default;
- 2 if a new calibration is required, rebuild the bootstrap (r, FAR) points from raw stored background features or start a new background file version;
- 3 store model version, fit metadata, and calibration metadata in every background file.

- 1 Merge main SPIIR and wguo-single-detector extraction into Eric branch.
- 2 Ensure `pipeline.sh -single` creates `sdpostcoh*.xml.gz` while coherent branch continues.
- 3 Install `single_detector_far.py` and `combine_background_far.py`.
- 4 Build/load `single_far_llr_background.json`.
- 5 Calibrate ν_{eff} from background rows in H1/L1 categories.
- 6 Compute LLR rank for all background and foreground single features.
- 7 If no FAR-LLR file exists, bootstrap direct FAR for about one week and fit $F_{\text{fit}}(r)$.
- 8 After the FAR-LLR file exists, assign FAR through $\text{FAR} = F_{\text{fit}}(r_*)$.
- 9 Write the single CSV with $\rho = \text{snglsnr}[j]$ and $y = -\log_{10} \text{FAR}_{\text{sd}}^{(j)}$.
- 10 Later merge with coherent products only after each branch has assigned calibrated FAR.

The completed single branch should produce rows like:

category	detector	ρ	χ^2	r	$-\log_{10} \text{FAR}$
H1_sd	H1	snglsnr[H1]	chisq[H1]	r_{H1}	$-\log_{10} \text{FAR}_{\text{H1}}$
L1_sd	L1	snglsnr[L1]	chisq[L1]	r_{L1}	$-\log_{10} \text{FAR}_{\text{L1}}$

This is the correct endpoint for the current study: a calibrated single-detector FAR plane. The raw feature plane (ρ_m, χ_r^2) is an intermediate; the final comparable quantity is $(\rho_m, -\log_{10} \text{FAR}_{\text{sd}})$.